

PRACTICAL 31 -

Objective - Dijkstra's Algorithm (Shortest Path)

Implement Dijkstra's algorithm to find shortest distances from a source node.

CODE -

```
1  #include <stdio.h>
2  #include <limits.h>
3  #include <stdbool.h>
4
5  #define MAX_NODES 5
6
7  // Function to find the vertex with the minimum distance value
8  int min_distance(int dist[], bool visited[], int numVertices) {
9      int min = INT_MAX, min_index;
10
11     for (int v = 0; v < numVertices; v++) {
12         if (!visited[v] && dist[v] <= min) {
13             min = dist[v];
14             min_index = v;
15         }
16     }
17     return min_index;
18 }
19
20 // Dijkstra's algorithm to find the shortest path
21 void dijkstra(int graph[MAX_NODES][MAX_NODES], int src, int numVertices) {
22     int dist[MAX_NODES]; // Output array to hold the shortest
23                          // distance from src to each vertex
24     bool visited[MAX_NODES]; // Boolean array to track visited
25                              // vertices
26
27     // Initialize distances as infinite and visited[] as false
28     for (int i = 0; i < numVertices; i++) {
29         dist[i] = INT_MAX;
30         visited[i] = false;
31     }
32
33     // Distance of source vertex to itself is always 0
34     dist[src] = 0;
35
36     // Find shortest path for all vertices
37     for (int count = 0; count < numVertices - 1; count++) {
38         // Pick the minimum distance vertex from the set of unvisited
39         // vertices
40         int u = min_distance(dist, visited, numVertices);
```

```
38
39     // Mark the picked vertex as visited
40     visited[u] = true;
41
42     // Update distance value of the adjacent vertices of the picked
    vertex
43     for (int v = 0; v < numVertices; v++) {
44         // Update dist[v] only if:
45         // 1. It's not visited
46         // 2. There's an edge from u to v (graph[u][v] > 0)
47         // 3. Total weight of path from src to v through u is
            smaller than the current value of dist[v]
48         if (!visited[v] && graph[u][v] && dist[u] != INT_MAX &&
            dist[u] + graph[u][v] < dist[v]) {
49             dist[v] = dist[u] + graph[u][v];
50         }
51     }
52 }
53
54 // Print the shortest distances
55 printf("Vertex\tDistance from Source\n");
56 for (int i = 0; i < numVertices; i++) {
57     printf("%d\t%d\n", i, dist[i]);
58 }
59 }
60
61 // Main function to demonstrate Dijkstra's Algorithm
62 int main() {
63     int numVertices = 5;
64     int graph[MAX_NODES][MAX_NODES] = {
65         {0, 10, 0, 30, 100},
66         {10, 0, 50, 0, 0},
67         {0, 50, 0, 20, 10},
68         {30, 0, 20, 0, 60},
69         {100, 0, 10, 60, 0}
70     };
71     int source = 0; // Source vertex
72
73     // Run Dijkstra's Algorithm
74     dijkstra(graph, source, numVertices);
75
76     return 0;
77 }
```

OUTPUT -

Vertex	Distance from Source
0	0
1	10
2	50
3	30
4	60